

Circuit Breaker & Resilience Patterns

Comprehensive Guide to Building Fault-Tolerant Microservices

Spring Boot 3.x + Resilience4j Implementation Guide

Project: FoodFlow — Food Delivery Backend

Architecture: Microservices (Spring Cloud)

Focus: Circuit Breaker, Retry, Bulkhead, Rate Limiter

Library: Resilience4j 2.1.0

Framework: Spring Boot 3.x

Table of Contents

1. What is Resilience in Microservices Architecture
2. Problems Caused by Cascading Failures
3. What Happens When a Dependent Service is Slow or Down
4. Introduction to Circuit Breaker Pattern
5. How Circuit Breaker Prevents System-Wide Failures
6. States of Circuit Breaker: CLOSED, OPEN, HALF-OPEN
7. Difference Between Retry and Circuit Breaker
8. Introduction to Resilience4j
9. Configuring Failure Threshold and Wait Duration
10. Implementing Fallback Methods
11. Monitoring Circuit Breaker Health
12. Real-World Examples of Resilience Patterns
13. Common Configuration Mistakes and Overuse of Retry

1. What is Resilience in Microservices Architecture

Resilience is the ability of a system to continue functioning correctly even when some of its components fail or experience degraded performance. In microservices architecture, resilience is critical because services are distributed and communicate over the network.

Why resilience matters in microservices:

- Services are distributed and communicate over the network
- Each service has its own failure domain
- A single service failure can cascade to dependent services
- Network latency and timeouts are inherent in distributed systems

Resilience patterns protect the system from:

- Service unavailability (down services)
- High latency (slow services)
- Network failures
- Resource exhaustion (thread pools, connections)

2. Problems Caused by Cascading Failures

Cascading failures occur when a failure in one service triggers failures in dependent services, creating a domino effect that can bring down the entire system.

Example scenario:

Payment Service slows down → Order Service threads block waiting
→ Order Service thread pool exhausts → API Gateway requests timeout
→ Customer retries → More requests flood the system
→ All services become unresponsive → System-wide outage

Common cascading failure patterns:

- Thread pool exhaustion from blocked calls
- Connection pool depletion from waiting connections
- Memory exhaustion from queued requests
- Database lock contention from long-running transactions
- Retry storms amplifying traffic to failing services

3. What Happens When a Dependent Service is Slow or Down

When a service is DOWN:

- Connection attempts timeout
- Feign clients throw `ConnectException` or `SocketTimeoutException`
- Calling service experiences increased latency
- If no timeout is set, threads block indefinitely

- Thread pool fills up, rejecting new requests

When a service is SLOW:

- Requests complete but with high latency
- Response times exceed configured timeouts
- Backpressure builds up in calling services
- Resource pools (threads, connections) get exhausted
- User experience degrades across the system

Impact on FoodFlow platform:

- Order Service calling slow Payment Service → orders hang
- Restaurant Service calling slow Menu Service → menu loading fails
- Delivery Service calling slow Notification Service → delivery updates delayed

4. Introduction to Circuit Breaker Pattern

The Circuit Breaker pattern prevents cascading failures by detecting when a dependent service is failing and temporarily blocking calls to it, similar to an electrical circuit breaker that trips when there's a fault.

Key benefits:

- Fast failure when a service is known to be down
- Prevents resource exhaustion on the calling side
- Allows the failing service time to recover
- Provides fallback behavior for users

How it works:

- 1. Monitor calls to a dependent service
- 2. Count failures over a sliding time window
- 3. When failure threshold is reached, trip the circuit (OPEN state)
- 4. Block all calls immediately (fail fast)
- 5. After a wait duration, attempt a single call (HALF-OPEN state)
- 6. If successful, reset to CLOSED; if failed, stay OPEN

5. How Circuit Breaker Prevents System-Wide Failures

Without Circuit Breaker:

```
Order Service → Payment Service (slow/down)
■■ Thread 1: waiting... (blocked)
■■ Thread 2: waiting... (blocked)
■■ Thread 3: waiting... (blocked)
■■ Thread pool exhausted → all new requests rejected
```

With Circuit Breaker:

```
Order Service → Payment Service (circuit OPEN)
■■ Thread 1: CircuitBreakerOpenException (immediate)
■■ Thread 2: CircuitBreakerOpenException (immediate)
■■ Thread 3: CircuitBreakerOpenException (immediate)
■■ Fallback: "Payment unavailable, try later" (graceful degradation)
```

Protection mechanisms:

- Immediate failure instead of waiting
- Resource conservation (threads, connections)
- Graceful degradation via fallbacks
- Automatic recovery detection

6. States of Circuit Breaker: CLOSED, OPEN, HALF-OPEN

CLOSED State (Normal Operation):

- Circuit allows all requests to pass through
- Failure rate is monitored but below threshold
- Each failed request increments failure counter
- Successful requests reset failure counter
- Example: Payment Service responding normally

OPEN State (Circuit Tripped):

- All requests are blocked immediately
- No actual calls to the dependent service
- `CircuitBreakerOpenException` thrown for each call
- Fallback methods are invoked
- Wait duration timer starts (e.g., 60 seconds)
- Example: Payment Service down, circuit tripped after 50% failure rate

HALF-OPEN State (Recovery Attempt):

- After wait duration expires, circuit transitions to HALF-OPEN
- Allows a single request through to test if service recovered
- If request succeeds → transition to CLOSED (circuit reset)
- If request fails → transition back to OPEN (wait duration restarts)
- Example: Testing if Payment Service recovered after outage

7. Difference Between Retry and Circuit Breaker

Aspect	Retry Pattern	Circuit Breaker Pattern
Purpose	Handle transient failures	Prevent cascading failures
When to use	Network glitches, temporary service	Service down, persistent failures
Behavior	Re-attempt the same request	Block requests to failing service
Resource usage	Increases load (more requests)	Decreases load (blocks requests)
Failure handling	Hope it works next time	Accept failure, use fallback
Example scenario	Payment gateway timeout (retry more)	Payment service down (stop calling)

When to use together:

- Retry first for transient failures (1-3 attempts)
- Circuit Breaker trips if retries consistently fail
- This combination handles both temporary and persistent failures

8. Introduction to Resilience4j

Resilience4j is a fault tolerance library for Java that implements several resilience patterns. It is the recommended replacement for Hystrix, which is now in maintenance mode.

Patterns provided by Resilience4j:

- **Circuit Breaker:** Prevent cascading failures
- **Retry:** Handle transient failures
- **Rate Limiter:** Control request rate
- **Bulkhead:** Limit concurrent calls
- **Time Limiter:** Cancel long-running operations
- **Cache:** Cache responses to reduce load

Why Resilience4j over Hystrix:

- Active maintenance (Hystrix is in maintenance mode)
- Java 8+ compatibility
- Modular design (use only what you need)
- Better performance
- Reactive programming support

Integration with Spring Boot:

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>2.1.0</version>
</dependency>
<dependency>
```

```

<groupId>io.github.resilience4j</groupId>
<artifactId>resilience4j-feign</artifactId>
<version>2.1.0</version>
</dependency>

```

9. Configuring Failure Threshold and Wait Duration

Configuration in application.yml (from config-server):

```

resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        slidingWindowSize: 10           # Number of calls in sliding window
        minimumNumberOfCalls: 5         # Minimum calls before calculating rate
        failureRateThreshold: 50        # Failure percentage to trip (50%)
        waitDurationInOpenState: 60s    # Time in OPEN before HALF-OPEN
        permittedNumberOfCallsInHalfOpenState: 3 # Test calls in HALF-OPEN
        slowCallRateThreshold: 100      # Slow call rate threshold
        slowCallDurationThreshold: 2s   # What qualifies as "slow"
        recordExceptions:
          - java.util.concurrent.TimeoutException
          - java.io.IOException
        ignoreExceptions:
          - com.foodflow.order.exception.BusinessException

```

Key parameters explained:

Parameter	Description
slidingWindowSize	Window size for failure rate calculation (count-based or time-based)
failureRateThreshold	Percentage of failures that triggers OPEN state
waitDurationInOpenState	How long to stay OPEN before attempting recovery
slowCallRateThreshold	Trip if calls are consistently slow (even if they succeed)
permittedNumberOfCallsInHalfOpenState	How many test calls to allow in recovery

Applying to Feign client:

```

@FeignClient(
  name = "payment-service",
  fallback = PaymentServiceClientFallback.class,
  configuration = FeignCircuitBreakerConfig.class
)
public interface PaymentServiceClient {
  @PostMapping("/api/v1/payments")
  ApiResponse<PaymentResponseDto> createPayment(@RequestBody PaymentRequestDto dto);
}

```

10. Implementing Fallback Methods

Fallback class for Feign client:

```
@Component
@Slf4j
public class PaymentServiceClientFallback implements PaymentServiceClient {

    @Override
    public ApiResponse<PaymentResponseDto> createPayment(PaymentRequestDto dto) {
        log.error("Payment service unavailable, using fallback for order: {}", dto.getOrderId());

        // Return a graceful response indicating payment service is down
        return ApiResponse.<PaymentResponseDto>error(
            "Payment service temporarily unavailable. Please try again later.",
            List.of("PAYMENT_SERVICE_UNAVAILABLE")
        );
    }
}
```

Fallback strategies:

- **Return cached data:** Use last known good value
- **Return default value:** Provide sensible defaults
- **Return error message:** Inform user of unavailability
- **Queue for later processing:** Store request, process when service recovers
- **Call alternative service:** Use backup provider

Programmatic Circuit Breaker with fallback:

```
@CircuitBreaker(
    name = "paymentService",
    fallbackMethod = "createPaymentFallback"
)
public PaymentResponseDto createPayment(PaymentRequestDto dto) {
    return paymentServiceClient.createPayment(dto).getData();
}

private PaymentResponseDto createPaymentFallback(PaymentRequestDto dto, Exception ex) {
    log.error("Circuit breaker triggered for payment service", ex);

    // Create a pending payment record to process later
    PaymentResponseDto fallback = new PaymentResponseDto();
    fallback.setOrderId(dto.getOrderId());
    fallback.setStatus("PENDING_RETRY");
    fallback.setMessage("Payment queued for retry");

    return fallback;
}
```

11. Monitoring Circuit Breaker Health

Metrics exposed by Resilience4j:

- Circuit state (CLOSED, OPEN, HALF-OPEN)
- Failure rate
- Success rate
- Number of buffered calls

- Number of failed calls
- Number of slow calls

Actuator endpoint integration:

```
management:
  endpoints:
    web:
      exposure:
        include: health,metrics,circuitbreakers
  endpoint:
    health:
      show-details: always
  metrics:
    export:
      prometheus:
        enabled: true
```

Accessing metrics:

```
GET /actuator/circuitbreakers
GET /actuator/metrics/resilience4j.circuitbreaker.state
GET /actuator/metrics/resilience4j.circuitbreaker.failure.rate
GET /actuator/prometheus
```

Sample Prometheus queries:

```
# Circuit breaker state (0=CLOSED, 1=OPEN, 2=HALF_OPEN)
resilience4j_circuitbreaker_state{name="paymentService"}

# Failure rate percentage
resilience4j_circuitbreaker_failure_rate{name="paymentService"}

# Number of successful calls
rate(resilience4j_circuitbreaker_successful_calls{name="paymentService"}[5m])
```

Logging circuit state changes:

```
@Component
@Slf4j
public class CircuitBreakerEventListener {

    @EventListener
    public void onCircuitBreakerEvent(CircuitBreakerOnStateTransitionEvent event) {
        log.warn("Circuit breaker state transition: {} from {} to {}",
            event.getCircuitBreakerName(),
            event.getStateTransition().getFromState(),
            event.getStateTransition().getToState());
    }
}
```

12. Real-World Examples of Resilience Patterns

Example 1: Order Service calling Payment Service

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
@Retry(name = "paymentService", fallbackMethod = "paymentFallback")
@TimeLimiter(name = "paymentService")
public CompletableFuture<PaymentResponseDto> processPayment(PaymentRequestDto dto) {
    return CompletableFuture.supplyAsync(() ->
        paymentServiceClient.createPayment(dto).getData()
    );
}

private CompletableFuture<PaymentResponseDto> paymentFallback(PaymentRequestDto dto, Exception ex) {
    // Queue payment for async processing
    paymentQueueService.enqueueForRetry(dto);

    return CompletableFuture.completedFuture(
        PaymentResponseDto.builder()
            .status("QUEUED")
            .message("Payment queued for processing")
            .build()
    );
}
```

Example 2: Menu Service with Bulkhead (limit concurrent calls)

```
@Bulkhead(name = "menuService", type = Bulkhead.Type.THREADPOOL)
public List<MenuItemResponseDto> getMenuItems(Long restaurantId) {
    return menuServiceClient.getMenuByRestaurant(restaurantId).getData();
}
```

Example 3: Rate Limiter on public API

```
@RateLimiter(name = "publicApi", fallbackMethod = "rateLimitFallback")
public ResponseEntity<ApiResponse<RestaurantResponseDto>> getRestaurant(Long id) {
    return ResponseEntity.ok(restaurantService.getRestaurantById(id));
}

private ResponseEntity<ApiResponse<RestaurantResponseDto>> rateLimitFallback(Long id, Exception ex) {
    return ResponseEntity.status(429).body(
        ApiResponse.error("Too many requests, please try again later", null)
    );
}
```

Example 4: Cache with Circuit Breaker

```
@Cacheable(value = "restaurants", key = "#id")
@CircuitBreaker(name = "restaurantService", fallbackMethod = "restaurantFallback")
public RestaurantResponseDto getRestaurant(Long id) {
    return restaurantServiceClient.getRestaurantById(id).getData();
}

private RestaurantResponseDto restaurantFallback(Long id, Exception ex) {
    // Try to get from cache even if circuit is open
    return cacheManager.getCache("restaurants").get(id, RestaurantResponseDto.class);
}
```

13. Common Configuration Mistakes and Overuse of Retry

Common mistakes:

Mistake	Impact	Fix
Setting failure threshold too high	Too many failures before circuit opens	Use 50-60% threshold
Wait duration too short	Circuit flips rapidly, doesn't give service time to recover	Use 20-30 seconds
No timeout on Feign clients	Threads block indefinitely on slow services	Allow service connect and read timeouts
Retry without exponential backoff	Retry storms overwhelm failing service	Use exponential backoff (1s, 2s, 4s, 8s)
Circuit breaker on non-critical paths	Unnecessary complexity	Use fallback only, no circuit breaker for non-critical services
Ignoring specific exceptions	Circuit never trips on real failures	Only ignore business exceptions, not system exceptions

Overuse of retry:

```
# BAD: Excessive retry configuration
resilience4j:
  retry:
    instances:
      paymentService:
        maxAttempts: 10          # Too many retries
        waitDuration: 100ms     # Too fast between retries
        retryExceptions:
          - java.lang.Exception  # Retry on everything

# GOOD: Balanced retry configuration
resilience4j:
  retry:
    instances:
      paymentService:
        maxAttempts: 3          # Reasonable limit
        waitDuration: 1s       # Give service time
        exponentialBackoffMultiplier: 2 # Exponential backoff
        retryExceptions:
          - java.net.SocketTimeoutException
          - java.io.IOException
        ignoreExceptions:
          - com.foodflow.payment.exception.InsufficientFundsException
```

When NOT to use retry:

- Non-idempotent operations (POST without idempotency key)
- Business logic failures (insufficient funds, invalid data)
- Authentication failures (wrong password)
- Rate limit exceeded (429 status)
- Validation errors (400 status)

Best practices summary:

- Use Circuit Breaker for all cross-service calls
- Use Retry only for transient network failures
- Always configure timeouts on Feign clients
- Implement meaningful fallbacks (not just null returns)
- Monitor circuit state and metrics
- Test failure scenarios in integration tests

- Don't over-engineer for non-critical paths

FoodFlow Platform — Circuit Breaker & Resilience Patterns Guide v1.0